CHAPTER 01 · OVERVIEW

How It All Fits Together

What runs where, what SLAM and Nav2 each actually do, and the workflow that gets from **manual driving** to **trusted autonomous navigation**. Read this chapter first.

STAGE 1

Map

- Drive manually from the dashboard
- slam_toolbox builds the map live
- All CPU goes to scan matching

`SLAM: on` `Nav2: off`

→

STAGE 2

Freeze

- Drive back to the physical zero mark
- Read the loop-closure error (the test)
- Save the map to a file, stop SLAM

`SLAM: off` `Nav2: off`

→

STAGE 3

Navigate

- Load the saved map (a static file)
- AMCL localizes the robot inside it
- Nav2 drives autonomously to goals

`SLAM: off` `Nav2: on`

You keep the map **in every stage**. Stage 3 doesn't throw anything away — it just stops letting Nav2 rewrite the map **while** it's driving. That's the only thing that changes.

Color key — used in every diagram below

hardware · manual control · sensing & odometry · SLAM / mapping · Nav2 / autonomy · custom fix (bug found on this project)

CHAPTER 02 · CONCEPTS

Three Words You Keep Hearing

ROS 2

THE MESSAGING SYSTEM

Not a robot brain. It's how independent programs (*nodes*) talk to each other: continuous streams (**topics**), one-off request/response (**services**), long tasks with progress updates (**actions**), and a live tree of "where is X relative to Y" (**TF**).

Doesn't: drive the robot, build a map, or plan a path — it just moves data between the nodes that do.

SLAM

VIA SLAM_TOOLBOX

Answers *"where am I, and what does the world look like?"* Reads the LiDAR, matches each new scan against everything seen so far, builds the occupancy-grid map, and corrects the robot's believed position when it recognizes somewhere it's been (**loop closure**).

Doesn't: drive the robot. It only perceives and localizes.

Nav2

THE NAVIGATION STACK

Answers *"how do I get from here to there without hitting anything?"* Given a goal and a map, it plans a route and continuously steers the robot along it, reacting to live obstacles as it goes.

Doesn't: build maps. It consumes one — live-growing or frozen, either works.

The distinction that's been causing the confusion

Both of these are valid ways to run SLAM + Nav2 together. They trade the same three things differently.

WHAT YOU'VE BEEN RUNNING

SLAM-while-driving

- slam_toolbox and Nav2 both run **live, at the same time**
- Every scan match can shift `map→odom` — the map's origin relative to the robot's short-term motion
- The 3D view "re-centers" every time that shift happens — **this is SLAM correcting itself, working as designed**, not a bug
- Costs the most CPU: scan matching and MPPI's ~500-trajectory search are fighting for the same Pi 5

Good for: exploring and building the first map. Rough for: a smooth, trustworthy autonomous drive.

WHAT'S RECOMMENDED NEXT

Map once, then localize

- Map is built once, **saved to a file**, and slam_toolbox is turned **off**
- AMCL — a lighter localizer — places the robot inside that fixed map using particle filtering, not live scan-matching
- Nothing can recenter, because nothing is allowed to rewrite the map anymore
- Frees real CPU headroom for Nav2's controller

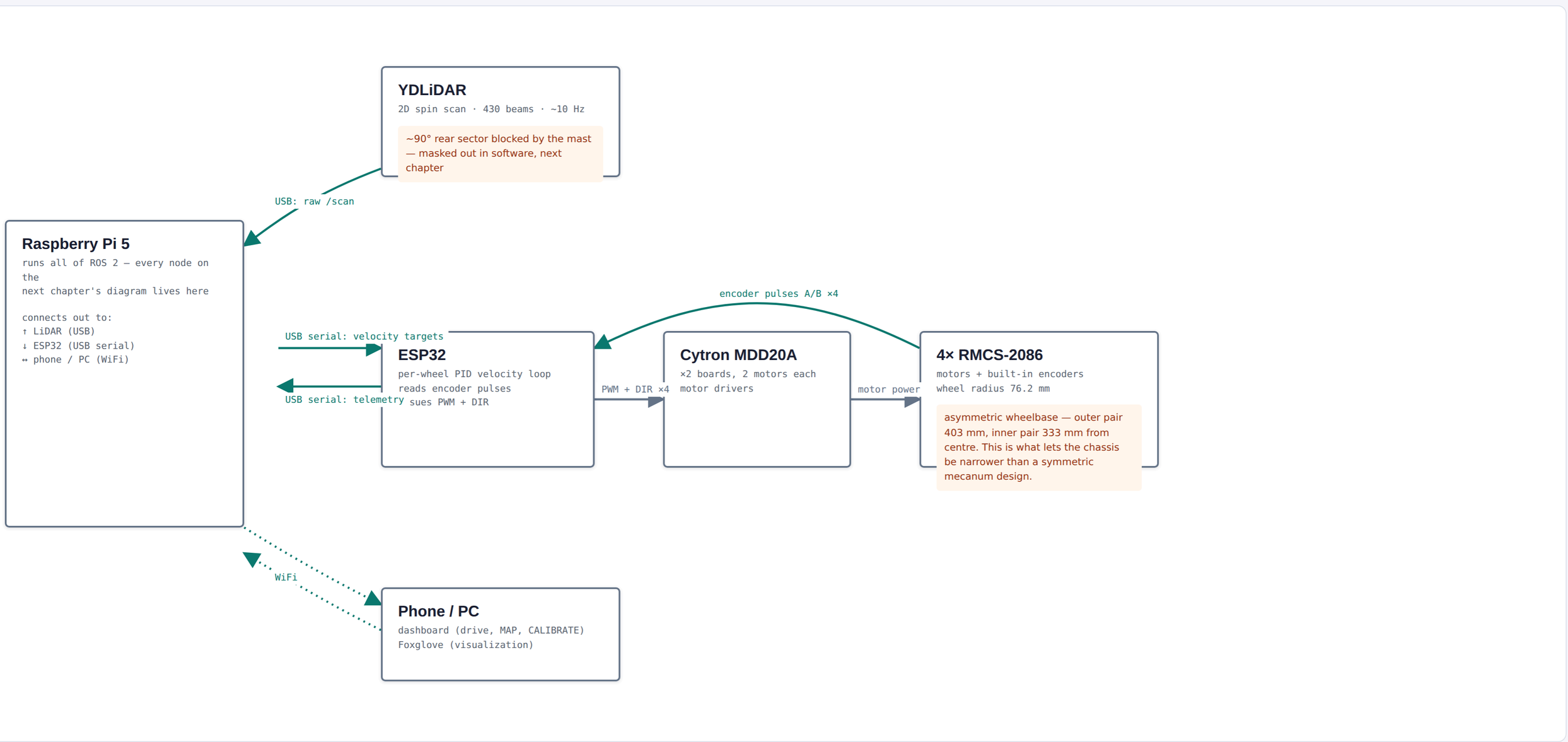
Good for: trustworthy, repeatable autonomous navigation once the map is solid.

Either way, the map is a file on disk. You never lose it. The only choice is whether Nav2 is allowed to keep editing that file while it's also trying to drive.

CHAPTER 03 · HARDWARE

The Physical Chain

Everything below is a real box, wire or radio link. Software starts in the next chapter.



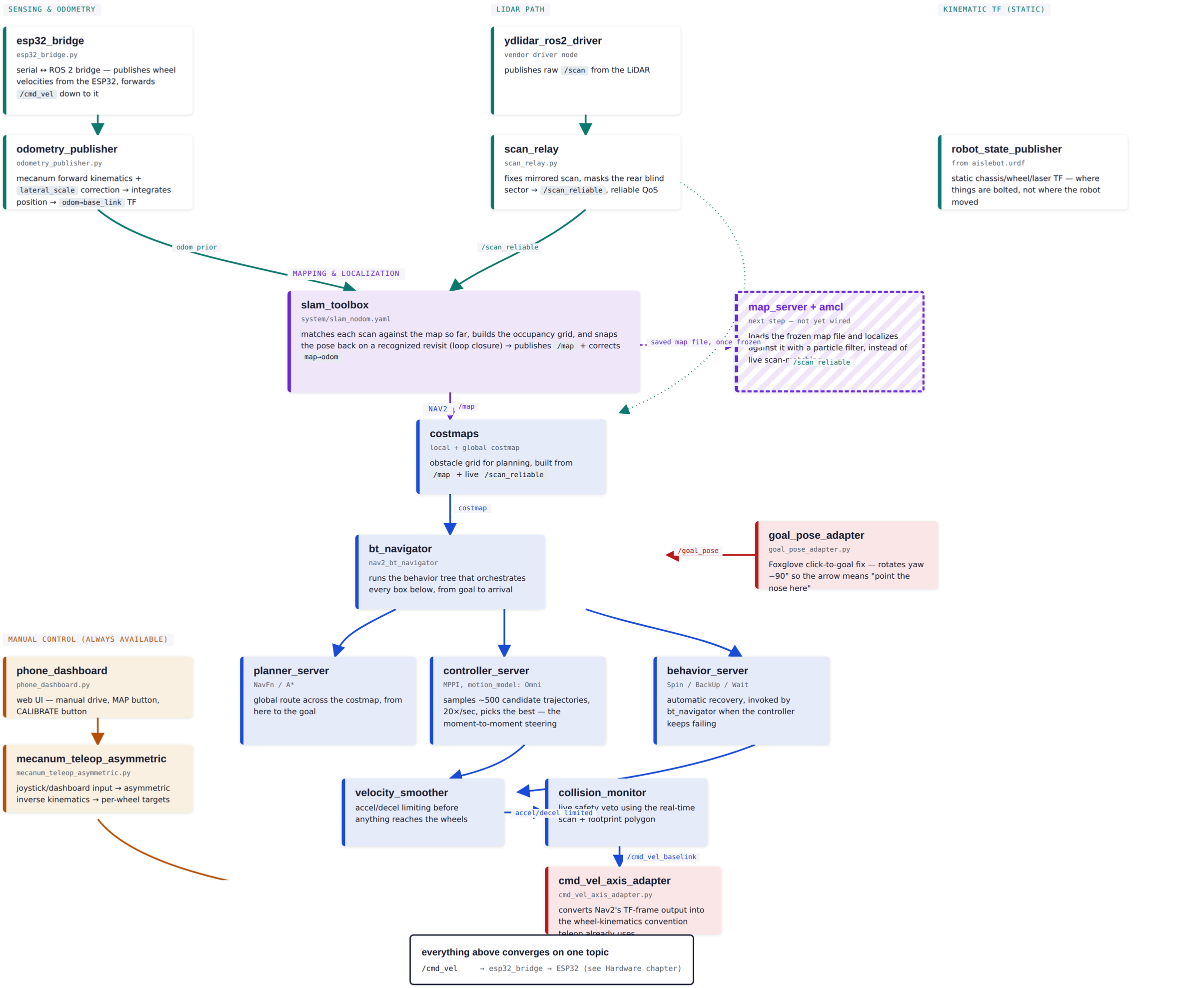
mechanical / power data / telemetry

Diagram is wider than this column — scroll sideways inside the box above if any of it is cut off.

CHAPTER 04 · SOFTWARE

The ROS 2 Node Graph

Every box is a separate program, talking to the others only through ROS 2 topics/TF (Chapter 02). Colors match the key in Chapter 01.



This is the big one — scroll sideways inside the box above to see the full graph.

CHAPTER 05 · STATUS

Where Things Actually Stand

Every line below is backed by a real hardware run or a real commit, not a plan. The right column is the specific remaining work to reach Stage 3 (Chapter 01) on a real, trustworthy map.

Built & confirmed on hardware

Confirmed means: measured on the robot, not just deployed.

- **Zero-point re-mark procedure**
  park on the physical mark, restart the SLAM service, TF reads (0,0,0) again
- **lateral_scale = 0.92 deployed & live**
  corrects mecanum roller-scrub over-reporting on strafe; confirmed via ros2 param get
- **LiDAR mirror + rear-occlusion fix**
  scan_relay.py corrects the flipped scan and masks the ~90° mast blind sector
- **Non-standard axis bugs found & fixed**
  this base_link is deliberately not REP-103 — repeatedly broke stock Nav2 defaults; each case root-caused and patched (goal_pose_adapter, cmd_vel_axis_adapter, MPPI critic choice)
- **DWB → MPPI controller switch**
  DWB's RotateToGoalCritic had a bit-exact rejection bug on this axis convention, read straight from source; MPPI's additive critic design can't reproduce it
- **MPPI validated: rotation & short translation**
  clean 45° spin (8.5mm centre drift, zero recoveries); 0.30m/0.50m runs landed exactly at the goal tolerance boundary
- **Two real Nav2 bringup bugs fixed**
  wait_for_service_timeout was in ms, not sec (was 5); inflation_radius was smaller than the robot's own circumscribed radius
- **Goal-preemption bug found & fixed**
  a client-side timeout wasn't cancelling the server-side goal, so the next command preempted it mid-drive — this was the "random" erratic motion on the first repeatability run
- **Data-collection tooling built**
  trajectory_viz.py (path recording + deviation stats) and repeatability_test.py (tape-measured multi-rep logging to CSV) — both used for APS data
- **Loop-closure / scan-matcher tuning written**
  slam_nodom.yaml given a real loop-closure block for the first time — not yet run on hardware `untested`

Still needed

In the order they unblock each other.

1. **Finish the manual, Nav2-off tape-measure test**
   straight / strafe / diagonal, believed position vs. physical tape, with Nav2 not running at all — the original clean baseline, started but not finished
2. **Drive one complete map, SLAM only, Nav2 off**
   full loop of the space with the new loop-closure tuning, ending back on the physical zero mark
3. **Confirm loop closure numerically**
   read zero_point→base_link on return — this is the actual pass/fail test for whether the tuning worked
4. **Save the map, stop SLAM**
   freeze it to a file — this is the moment the map stops being editable, per Chapter 01
5. **Wire up map_server + AMCL**
   the dashed box in Chapter 04 — not written yet, no launch config exists in the repo
6. **Re-validate MPPI against the frozen map**
   rotation + translation results so far were all measured while SLAM was still live — repeat under Stage 3 conditions
7. **Confirm the CPU-starvation crash is actually gone**
   MPPI + live SLAM + costmap resizing froze the LiDAR pipeline outright on one run and caused a collision — AMCL should be far cheaper, but this needs to be measured, not assumed
8. **Full blended validation**
   straight / strafe / diagonal / rotation trusted individually — never yet combined into one real goal-to-goal run

The next hardware session, in order

01 **Drive the map.** SLAM on, Nav2 off, full loop back to the mark.

02 **Read the return error.** zero_point→base_link should now be near (0,0).

03 **Save + freeze.** Map to file, kill slam_toolbox.

04 **Bring up AMCL.** Load the static map, localize, no recentering possible.

05 **Re-run Nav2 goals** and confirm the CPU headroom fixes the freeze/collision failure.